Ludo like UCSC

Sanuth Wijayarathna

September 2024

1 Introduction

LUDO is a board game for two (2) to four (4) players inspired by the ancient Indian cross and circle board game Pachisi. Ludo differs from Pachisi by including six (6) sided dice to make moves on the Ludo board.

I have made the traditional Ludo game and extended it by applying counter clockwise and mystery cells, although I couldn't add player behaviours and blocks. I divided the code into three files named `main.c`, `logic.c`, `logic.h`, and `types.h`.

2 `types.h`

This file has the types and constants used for the Ludo game. It sets up the game board, player properties, and special locations with specific behaviours.

- **BOARD SIZE**: The total number of cells on the board (52 cells in standard Ludo).
- **PLAYERS**: Number of players in the game (4).
- **PIECES PER PLAYER**: Number of pieces each player controls (4).
- **HOME STRAIGHT START**: A constant indicating the position of the home straight (final stretch before reaching home) relative to the board positions.
- **CLOCKWISE** and **COUNTER CLOCKWISE**: Values indicating the direction of movement for pieces (1 for clockwise and -1 for counterclockwise).
- **CLOCKWISE CELLS TRAVELLED TO HOME** and **COUNTER CLOCKWISE CELLS TRAVELLED TO HOME**: Constants representing the total number of cells a piece needs to travel to reach home in either direction.
- **BHAWANA**, **KOTUWA**, **PITAKOTWA**: Specific cells on the board with special rules when a piece lands on them via the mystery cell.

- **YELLOWAPP**, **BLUEAPP**, **REDAPP**, **GREENAPP**: Constants representing the approach positions for each color on the board.

Above is the constants I used and why I used those numbers as constants

I made a struct called player and defined the structure with the arrays I needed for making the game.

enum color : This stores the colour of the players as an enum, for readability and easy access.
position: this is the original location of a single piece.
direction : this is the direction of a single piece (clock wise or counter clockwise).
cellstravelled : this is the cells travelled from the starting point of a single piece in the first round or from the approach cell in the second round.
piecescaptured : this is the array that stores the captured number of pieces of every player's pieces.
effectType : this comes under the bhawana effects, if a piece is affected an effect from bhawana it stores under this category.
effectRoundsLeft : Also under the bahwana effect, rray storing how many rounds are left for the current effect on each piece
roundsPassAppraoch : this is the array that stores the rounds a piece has passed the approach of that piece.
roundsLeftInKotuwa : Array storing how many rounds are left in kotuwa for each piece,
consecutiveThrees : this is only for kotuwa effect from the mystery cell, Array storing the number of consecutive threes rolled by the player

efficiency of types.h
The structure uses a single block of contiguous memory, which might be efficient for a number of reasons regarding access and management. The contiguous memory layout also helps cache performance as related data is likely to get loaded into the cache all at once.

Data Integrity: The structure allows integrity of data, as it allows all the related information to be stored together rather than dealing with a variety of independent variables that may lead to inconsistency in data.

Macros in C are a powerful feature provided by the preprocessor that can be used to define constants, inline functions, and more. They are a key tool for enhancing code readability. also in this file, an enumeration was used because it improves readability, error reduction and code maintainability.

I also added short for int types to save memory.

3 `logic.h`

This file mainly contains the of the ludo game. all the functions and initialization are done in the logic.c file. By making this file,

3.0.1 Encapsulation and Abstraction

- **Function Declarations**: Header files allow you to declare functions without defining them. This separation of declaration and definition helps in encapsulating the function interface from its implementation.
- **Abstraction**: By including function declarations in a header file, you provide a clear interface for other parts of your program or other programs to use the functions, without exposing the implementation details.

3.0.2 Code Organization

- **Modularity**: Header files help in organizing code into logical modules. Each header file typically corresponds to a specific module or set of related functions, making the codebase easier to navigate and manage.
- **Separation of Concerns**: Functions can be defined in separate `.c` files, while their declarations are included in header files. This separation helps in maintaining a clean structure and focusing on different aspects of the code.

3.0.3 Re-usability

- **Multiple Includes**: Header files allow you to reuse function declarations across multiple source files. By including the same header file in different `.c` files, you ensure that the function declarations are consistently available wherever they are needed.
- **Avoid Redefinition**: By including function declarations through header files, you avoid having to redeclare functions multiple times, which reduces redundancy and the chance of errors.

3.0.4 Ease of Maintenance

- **Centralized Updates**: When a function's declaration changes (e.g., due to a change in parameters), you only need to update the header file. All source files that include the header file will automatically be updated with the new declaration.
- **Consistency**: Keeping function declarations in header files ensures that all parts of the program use the same function signatures, which helps prevent inconsistencies and errors.

3.0.5 Compile-Time Checking

- **Type Safety**: Header files provide a way for the compiler to check function calls against their declarations. This compile-time checking helps catch errors such as mismatched function signatures or incorrect usage of functions early in the development process.
- **Dependency Management**: The compiler can ensure that all dependencies are met, and functions are called with the correct arguments.

4 `logic.c`

This is where all the logic of the game is stored. Every function, Every global variable and Every intialization is in here.

player structure intialization: This initialization sets up an array of 4 players, each with their attributes correctly initialized. This setup is essential for starting a game or simulation where each player's state needs to be defined before gameplay begins.

All players start with the same initial state, which is crucial for ensuring a consistent starting point for each player in the game.

I also used an array of 4 Player structures and made arrays of 4 in it because its easier to point out a single piece in the ludo game. ex : players[0].postion[0] (this means the first rolling player's first piece)

short int rolledvalue[4]; This is to store the rolled value of each player in a single round
short int rounds = 0; : store the rounds of the game
short int mystery cell = -1; : store the mystery cell number
short int onBase[4] = 4,4,4,4; : store the pieces on the base of each player

void gameintro() This is just a function to show the output of players,pieces and its colours and names

short int rando() This is the function generate random numbers from 1 to 6 which is needed to decide the number a player rolls and to decide which mystery cell a player teleport to.

void findmax(short int rolledvalue[], struct Player players[], short int n) This is the function which finds the maximum number rolled player. I used a while loop for which if there are two or more players who has the highest value, it continues till only one has the highest and to find the highest value i used the bubble sort algorithm method. Then used a switch case to determine the player order. for efficiency and time complexity and readability i used minimum loops and switch cases.

void roll(short int rolledvalue[], struct Player players[]) Then i made this roll function. The code simulates a game where players take turns rolling

dice and moving pieces on a board. It starts by initializing a round counter and setting up an array to track the order in which players finish, along with a count of how many players have completed the game. The main loop runs for a predefined number of rounds, incrementing the round counter and printing the current round number. For each player, the code checks if they have already finished the game and skips their turn if so. If the player's pieces are not in any special state, the dice are rolled, and the result is processed to determine movement, handling consecutive dice rolls of a specific value as needed. If a piece lands on the board, it is moved accordingly based on the dice result. The code also checks for captures of opponent pieces, allowing the player to roll again if a capture occurs. It then verifies if all of the player's pieces have reached their final position, records the finishing order, and updates the count of finished players. If the number of finished players reaches a certain threshold, the game ends. The player's turn is managed based on the dice roll and any special conditions, with the state of all players and special game elements being printed regularly to reflect the current status.

void movePieceToBoard(short int currentplayer, struct Player* players) Then made this function because after a roll if and only if a piece is on the board a move can be done.The function moves a piece from the base to the starting position on the board for a given player. It iterates through the player's pieces to find one that is still on the base. Once a piece is found, it is moved to the appropriate starting position based on the player's color, and a message is printed to indicate this move. The function then updates the count of pieces on the base and calculates the number of pieces currently on the board. It also selects a direction for the piece and prints the updated status. The function then exits after moving a single piece. In terms of efficiency, the function has a time complexity of O(P), where P is the number of pieces per player, as it checks each piece once until it finds one on the base. The operations performed inside the loop are constant time, making the function efficient for its purpose.

void movePieceOnBoard(short int currentPlayer, short int roll, struct Player* players)

Then i made this function which mainly handles the moves inside the board. The function moves a piece on the board for a given player based on a dice roll. It starts by determining the starting and approach positions based on the player's color. The function then iterates through the player's pieces, skipping those that are finished or on the base. It handles pieces in the home straight by checking if the rolled value allows them to reach home. If a piece is under a special effect, such as being energized or sick, the roll is adjusted accordingly, and the effect is updated. The function then moves the piece either clockwise or counterclockwise based on its current direction, adjusting its position to ensure it wraps around the board correctly. After updating the piece's position, the function tracks the cells traveled, checks for special conditions like reaching the home straight or encountering a mystery cell, and then exits after processing one piece. The time complexity of this function is O(P), where P is the number

of pieces per player, due to the single loop iterating through the pieces. The efficiency is generally high because the operations inside the loop are constant time, and the function ensures that only necessary checks and updates are performed.

void cellstravelledMysterycell(short int currentPlayer, short int pieceindex, struct Player* players, short int cellnum)

This function focuses on calculating the cells travalled of a piece. it helps to calculate how much cells a piece needs to go to enter the home straight. The function is also configured to change whenever a piece lands on a mystery cell as well. The time complexity is O(1), as it involves a constant number of operations regardless of the input size, and the efficiency is high due to the direct mapping of cell types and directions to their corresponding values.

checkAndCapturePiece(short int currentPlayer, struct Player* players)

the `checkAndCapturePiece` function is designed to identify and handle scenarios where a player's piece lands on the same square as an opponent's piece, resulting in a capture. The function starts by initializing a flag (`captureOccurred`) to track whether any captures happen during the execution. It then iterates over each piece of the current player to check its position on the board, excluding any pieces that are not actively in play (positions greater than -1).

For each piece on the board, the function proceeds to compare its position with the positions of all other players' pieces. When a match is found, indicating that the current player's piece has landed on the same square as an opponent's piece, several actions are triggered to simulate the capture: the opponent's piece is moved back to the base (position set to -1), its distance traveled (`cellstravelled`) and direction are reset, and the count of pieces captured for the opponent is also reset. The count of pieces on the base for the opponent is updated to reflect the capture, and the number of pieces the opponent has on the board is recalculated.

The function provides feedback through a series of `printf` statements, informing the players of the capture event, including details of which pieces were involved and the resulting positions. It also updates the count of captured pieces for the current player, reflecting their success in capturing an opponent's piece. Finally, the `captureOccurred` flag is set to `1` to indicate that a capture took place, and the function exits the inner loop to avoid redundant checks. After all possible checks, the function returns the `captureOccurred` flag, signaling to the calling function whether any captures occurred during this turn. This mechanism introduces a competitive element, encouraging strategic movement and positioning on the board.

The mystery cell functions are as follows

4.0.1 `cellstravelledMysterycell` Function

The `cellstravelledMysterycell` function updates the position and cells traveled for a player's piece when it lands on a mystery cell. This function first resets the number of rounds passed since the piece approached the home area. It then checks the current direction of the piece (either clockwise or counterclockwise) and the type of mystery cell encountered (`cellnum`). Depending on the mystery cell type and the player's color, it calculates a new position by adjusting the `cellstravelled` value accordingly. For example, landing on the Bhawana mystery cell (cell number 1) sets different travel distances based on the piece's color. This function is crucial for tracking each piece's movement dynamically and accurately when special cells alter the game's flow.

4.0.2 `kotuwa` Function

The `kotuwa` function manages the game's mechanics when a piece is teleported to Kotuwa, a specific cell where pieces are temporarily detained. When invoked, this function places the piece at the Kotuwa position, which is the 27th cell from the yellow approach, and sets the rounds remaining for the piece to stay in Kotuwa to four. Additionally, it resets the count of consecutive rolls of three for that piece, ensuring that previous rolls do not affect the new state. This function also provides feedback to the player by printing a message that indicates the piece's attendance at a briefing in Kotuwa and its inability to move for the next four rounds. It is designed to enforce the game's rules for pieces teleported to Kotuwa, providing a strategic element by pausing the piece's progression temporarily.

4.0.3 `pitakotuwa` Function

The `pitakotuwa` function is responsible for handling the teleportation and direction change when a piece is moved to Pita-Kotuwa. This function first checks the current direction of the piece. If the piece is moving clockwise, it changes its direction to counterclockwise and sets its new position to the Pita-Kotuwa cell, which is the 46th cell from the yellow approach. If the piece is already moving counterclockwise, it directly moves to Kotuwa instead of Pita-Kotuwa. The function then calls `cellstravelledMysterycell` to update the cells traveled, ensuring consistency in the game state. This function integrates the directional mechanics into the gameplay, adding complexity and strategic depth as players must adapt to changes in their piece's movement direction.

4.0.4 `bhawana` Function

The `bhawana` function manages the effects when a piece is teleported to the Bhawana cell. Upon reaching Bhawana, the piece is moved to the 9th cell from the yellow approach, and a random effect is applied to the piece: it either becomes "energized," doubling its movement speed, or "sick," halving its movement speed for the next four rounds. This function sets the effect type

(`energized` or `sick`) and initializes the duration of this effect to four rounds, providing immediate feedback to the player by printing a message about the new state of the piece. This randomness introduces an element of chance into the game, encouraging players to strategically plan for or react to unexpected changes.

4.0.5 `updateMysteryCell` Function

The `updateMysteryCell` function determines and updates the position of the mystery cell on the board. It first initializes an array to track occupied cells and iterates over all players and their pieces to mark which cells are currently occupied. Then, it compiles a list of all unoccupied cells into another array, from which a new position for the mystery cell is randomly selected. The position of the mystery cell is updated if the count of available cells is greater than zero. This function is called periodically (every four rounds, after the second round) to introduce variability in the game, ensuring that the mystery cell remains a dynamic and unpredictable element of gameplay.

4.0.6 `mysterycellcheck` Function

The `mysterycellcheck` function is the primary handler that checks whether a piece has landed on a mystery cell and applies the appropriate effects. If it's time to update the mystery cell, it calls `updateMysteryCell` to potentially change its position. If a piece lands on the mystery cell, a random number generator determines the type of mystery cell effect, such as teleporting to Bhawana, Kotuwa, Pita-Kotuwa, Base, X, or the approach. The function then calls the appropriate handling function based on the chosen effect, applying any special conditions or penalties to the piece. It provides feedback to the player by printing messages about the teleportation and any effects incurred. This function encapsulates the decision-making and randomness associated with mystery cells, adding an unpredictable element that requires players to adapt their strategies.

Each function plays a critical role in handling the interactions and effects of mystery cells, enhancing the complexity and excitement of your Ludo game while maintaining efficient execution and minimal memory use.

5 `main.c`

The `main()` function in my program is where the execution begins. It starts by seeding the random number generator with the current time using `srand(time(NULL));`. This ensures that each time the program runs, any random numbers generated (such as dice rolls) will be different, making the game unpredictable and fair. After setting up the randomization, the `game()` function is called, which contains the core logic for running the entire game—from player turns to determining the winner. Finally, the program ends by returning `0`, signaling that it has

completed successfully. This simple structure effectively initializes and runs the game in a controlled and repeatable manner.